ASSIGNMENT-2

Course: Full Stack Web Development
Course Code: 23CM4121
Assignment No.: 2
Student Name: N.Nandu
Roll No.: A24126552100
Date: 05-10-2026

Title

MongoDB Student Management System

Aim

Develop a simple MongoDB-based student management system to store, retrieve, update, and manage student information.

Objectives

1. To understand the use of MongoDB as a NoSQL document database.
2. To create and use a database and collection for student records.
3. To perform retrieval, conditional search, update, delete and sorting operations.
4. To create an index and demonstrate index usage.
5. To execute real-time queries on student information.

Theory

MongoDB is a NoSQL document database in which data is stored as documents inside collections. In this assignment, the database collegeDB and the students collection are used to manage student information. MongoDB Shell (mongosh) commands are used to retrieve, update, sort and index records.

Database Details

Database Name: collegeDB

Collection Name: students

Student Fields: rollNo, name, branch, year, marks, email

Technologies Used: MongoDB, MongoDB Shell (mongosh), VS Code

Operations Performed

1. Insert student records
2. Display all students
3. Display students by branch
4. Find students scoring more than 75
5. Search student using rollNo
6. Search students using conditions
7. Update student marks
8. Update student email
9. Delete student using rollNo
10. Sort students by marks
11. Create index on rollNo
12. Demonstrate index usage

Real-Time Queries

• Find students scoring above 80

• Find students scoring below 50

• Find the highest-scoring student

• Find students belonging to a particular branch

• Display students sorted according to marks

Program / Code

```
// MongoDB Student Management System

// Select database
use collegeDB;;

// Create collection if it does not exist
if (!(b.getCo.lectionNames().includes("s(udents"))
    {().("students")){ db.createCollection("stude(ts");"students");
}

// Display all students
db.students.fi.d().pretty();().();

// Display students from CSE-AIML
db.students.fi.d({ branch: "CSE(AIML" }).pretty(){:"CSE-AIML"}).();

// Students scoring more than 75
db.students.fi.d({ marks: { $gt( 75 } }).pretty(){:{$gt:75}}).();

// Search using rollNo
db.students.fi.d({ rollNo: "23C(001" }).pretty();{:"23CM001"}).();

// Students in 3rd year
db.students.fi.d({ year: 3 }).p(etty();{:3}).();

// Students scoring above 80
db.students.fi.d({ marks: { $gt( 80 } }).pretty(){:{$gt:80}}).();
```

Program / Code (continued)

```
// Students scoring below 50
db.students.fi.d({ marks: { $lt( 50 } }).pretty(){:{$lt:50}}).();

// Update marks
db.students.up.ateOne((
    { rollNo: "23:M001" },"23CM001"},
    { $set:$setarks: 90 }:}{:90}}
);;

// Update email
db.students.up.ateOne((
    { rollNo: "23:M001" },"23CM001"},
    { $set:$setmail: "rav:.kumar@examp{e.com" } }:"ravi.kumar@example.com"}}
);;

// Sort students by marks
db.students.fi.d().sort({ marks( -1 }).pretty();).({:1}).();

// Create index
db.students.cr.ateIndex({ rollNo: 1 })({:1});

// Display indexes
db.students.ge.Indexes();();

// Highest-scoring student
db.students.fi.d().sort({ marks( -1 }).limit(1).p)etty();.({:1}).(1).();
```

Program / Code (continued)

```
// Demonstrate index usage
db.students.fi.d((
    { rollNo: "23:M001" }"23CM001"}
)..xplain("e(ecutionSta"executionStats");
```

Algorithm / Procedure

Step 1

Start MongoDB and open MongoDB Shell (mongosh).

Step 2

Select the database using: use collegeDB

Step 3

Create the students collection if it does not already exist.

Step 4

Insert student records into the students collection.

Step 5

Display all students and perform branch, rollNo, year and marks searches.

Step 6

Update student marks and email using updateOne().

Step 7

Delete a student record using deleteOne().

Step 8

Sort student records by marks in descending order.

Step 9

Create an index on rollNo and display the indexes.

Step 10

Demonstrate index usage with explain("executionStats").

Step 11

Run the real-time queries: above 80, below 50, highest score and branch.

Step 12

Verify the outputs and record the results.

Test Cases and Execution Data

Test Case	Operation	Input / Query	Expected Result
TC01	Find	db.students.find()	Student records displayed
TC02	Branch	branch: CSE-AIML	Matching branch records displayed
TC03	Marks	marks > 75	Matching records displayed
TC04	Update	rollNo 23CM001	Student record updated
TC05	Index	createIndex({rollNo:1})	Index created
TC06	Highest	sort + limit(1)	Highest-scoring student displayed

Execution Data

The MongoDB commands were executed using the students collection. The screenshots stored in the repository correspond to the operations listed below and are represented in the output section.

OUTPUT

01. 01-display-all.png.png

```
db.students.find().pretty();
```

MongoDB Shell output: all student documents are displayed.

02. 02-branch-search.png.png

```
db.students.find({ branch: "CSE-AIML"
}).pretty();
```

MongoDB Shell output: students from CSE-AIML are displayed.

03. 03-marks-above-75.png.png

```
db.students.find({ marks: { $gt: 75 }
}).pretty();
```

MongoDB Shell output: students with marks greater than 75 are displayed.

04. 04-rollno-search.png.png

```
db.students.find({ rollNo: "23CM001"
}).pretty();
```

MongoDB Shell output: the matching rollNo document is displayed.

OUTPUT

05. 05-condition-search.png.png

```
db.students.find({ year: 3 }).pretty();
```

MongoDB Shell output: students satisfying the condition are displayed.

06. 06-update-marks.png.png

```
db.students.updateOne({ rollNo: "23CM001" },
{ $set: { marks: 90 } });
```

MongoDB Shell output: the marks update operation is executed.

07. 07-update-email.png.png

```
db.students.updateOne({ rollNo: "23CM001" },
{ $set: { email: "ravi.kumar@example.com" }
});
```

MongoDB Shell output: the email update operation is executed.

08. 08-delete.png.png

```
db.students.deleteOne({ rollNo: "23CM002" });
```

MongoDB Shell output: the matching student record is deleted.

OUTPUT

09. 09-sorting.png.png

```
db.students.find().sort({ marks: -1
}).pretty();
```

MongoDB Shell output: student documents are sorted by marks in descending order.

10. 10-index.png.png

```
db.students.createIndex({ rollNo: 1 });
```

MongoDB Shell output: the rollNo index is created.

11. 11-index-explanation.png.png

```
db.students.find({ rollNo: "23CM001"
}).explain("executionStats");
```

MongoDB Shell output: execution statistics demonstrate index usage.

12. 12-above-80.png.png

```
db.students.find({ marks: { $gt: 80 }
}).pretty();
```

MongoDB Shell output: students scoring above 80 are displayed.

OUTPUT

13. 13-below-50.png.png

```
db.students.find({ marks: { $lt: 50 }
}).pretty();
```

MongoDB Shell output: students scoring below 50 are displayed.

14. 14-highest-score.png.png

```
db.students.find().sort({ marks: -1
}).limit(1).pretty();
```

MongoDB Shell output: the highest-scoring student is displayed.

15. 15-branch.png.png

```
db.students.find({ branch: "CSE-AIML"
}).pretty();
```

MongoDB Shell output: branch-specific student records are displayed.

16. 16-final-sorting.png.png

```
db.students.find().sort({ marks: -1
}).pretty();
```

MongoDB Shell output: the final student list is displayed in sorted order.

Real-Time Queries

Above 80

```
db.students.fi.d({ marks: { $gt( 80 } }).pretty(){:{$gt:80}}).();
```

Below 50

```
db.students.fi.d({ marks: { $lt( 50 } }).pretty(){:{$lt:50}}).();
```

Highest Score

```
db.students.fi.d().sort({ marks( -1 }).limit(1).p)etty();.({:1}).(1).();
```

Particular Branch

```
db.students.fi.d({ branch: "CSE(AIML" }).pretty(){:"CSE-AIML"}).();
```

Sorted by Marks

```
db.students.fi.d().sort({ marks( -1 }).pretty();).({:1}).();
```

Result

Thus, the MongoDB Student Management System was successfully implemented. The required retrieval, conditional search, update, delete, sorting, indexing and real-time query operations were performed using MongoDB.

Conclusion

The assignment provides practical understanding of MongoDB databases, collections, documents, query operators, update operations, sorting and indexes. The students collection can be used to efficiently manage and retrieve student information.